SWE30003 - Journal

Table of Contents

1. Software Development Life Cycle and Requirements 4
   1.1. Verification v.s. Validation 4
   1.2. Types of requirements: 5
      1.2.1. Requirements 5
      1.2.2. Non-functional requirements 5
   1.3. Requirements Engineering 5
      1.3.1. Requirements Elicitation 6
      1.3.2. Requirements Analysis 7
2. Software design 7
3. Abstractions and Models 8
   3.1. Models 8
   3.2. Abstractions 8
   3.3. Models in abstractions 8
4. Scenarios and use cases 9
   4.1. User perspective 9
   4.2. Use cases 9
   4.3. Actors 10
5. Goal-Design Scale (unclear why a scale, but ok) 10
6. Domain Model Development 10
7. Task Descriptions 10
8. Workflows 11
9. Object Oriented Design 11
   9.1. Objects and classes 11
   9.2. Object-Oriented Design Process 12

9.3. Finding classes 13
9.3.1. CRC Cards 13
9.4. Responsibilities and Assigning Responsibilties 14
9.4.1. What are responsibilities? 14
9.4.2. Assigning Responsibilities 14
9.5. Relationships between classes 14
9.5.1. Example relationships 14
9.5.2. Association and Aggregation (and Composition) 15
9.5.3. Collections 15
9.6. Inheritance and Abstract Classes 16
9.6.1. Inheritance 16
9.6.2. Abstract Classes 16
9.6.3. How to design abstract classes 16
9.7. Colaborations 16
9.7.1. Roles and Protocols 17
9.8. Design by Contract 17
9.8.1. Abstract State v.s. Concrete State. 18
9.8.2. Command-Query seperation principle. 18
9.9. Decomposition 19
9.9.1. Functional v.s. Object-oriented Decomposition 20
9.10. Coupling and Cohesion 20

List of figures

Figure 1 Software Development Life Cycle 4
Table 1 Requirement Types 5
Figure 2 Requirements Engineering process 6
Figure 3 Software Design Process 7

Figure 4 Sample UML diagram 12
Table 2 Example CRC Card 13
Table 3 Some kinds of relationships between classes 15
Table 4 Command-Query Seperation Principle 18
Figure 5 Decomposition Process 19
Table 5 Functional v.s. Object-oriented Decomposition 20
Table 6 Coupling and Cohesion 20

1. Software Development Life Cycle and Requirements

The Software Development Life Cycle (SDLC) is a structured framework used by software organizations to design, develop, and test high-quality software. It defines the entire process of software production, from the initial idea to the final deployment and maintenance.

The SDLC is described below:

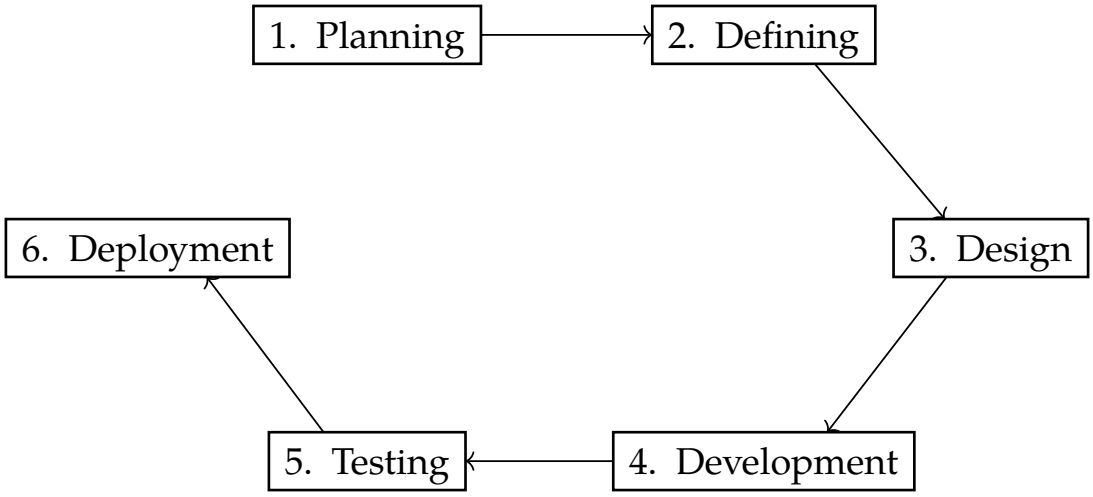

Figure 1: Software Development Life Cycle

In planning, the team must define what they are building and why (S1). Then, the specific product requirements must be defined and documented (S2).

1.1. Verification v.s. Validation

Verification is the process of **ensuring** that the software work products (such as requirements, design, and code) **conform to specified requirements and standards**. It focuses on **checking correctness and consistency** without necessarily executing the software. Verification asks: "Are we **building the product right**?"

Validation is the process of **evaluating the software** during or after development to **ensure it fulfills its intended use** and meets user and stakeholder needs. It typically involves executing the software and observing its behavior in real or simulated environments. Validation asks: "Are we **building the right product**?"

1.2. Types of requirements:

Only functional v.s. non-functional

1.2.1. Requirements

- **Functional requirements** are requirements that describe **system services** and **functions**
- **Non-functional requirements** (a.k.a **quality requirements**) are **constraints** on the services and/or the development process
- **Domain requirements** stem from application domain of a system, which might be functional/non-functional

1.2.2. Non-functional requirements

There are various types of non-functional requirements:

Requirement Type	Definition	Example
Product Requirement	Specify how the system should behave or perform (e.g. speed, reliability, functionality)	`latexmk` must call the LaTeX compiler enough times to resolve all references
Organizational Requirement	Requirements derived from organizational policies, standards, or processes	The system must conform to the process defined in an internal policy document
External Requirement	Requirements arising from external factors such as laws, regulations, or interoperability needs	The system must allow users to check whether their personal data is stored, in compliance with data protection laws

Table 1: Requirement Types

1.3. Requirements Engineering

Requirements Engineering is the process of identifying, eliciting, analyzing, specifying, validating, and managing the needs and expectations of stakeholders for a software system.

The requirements engineering process is as following:

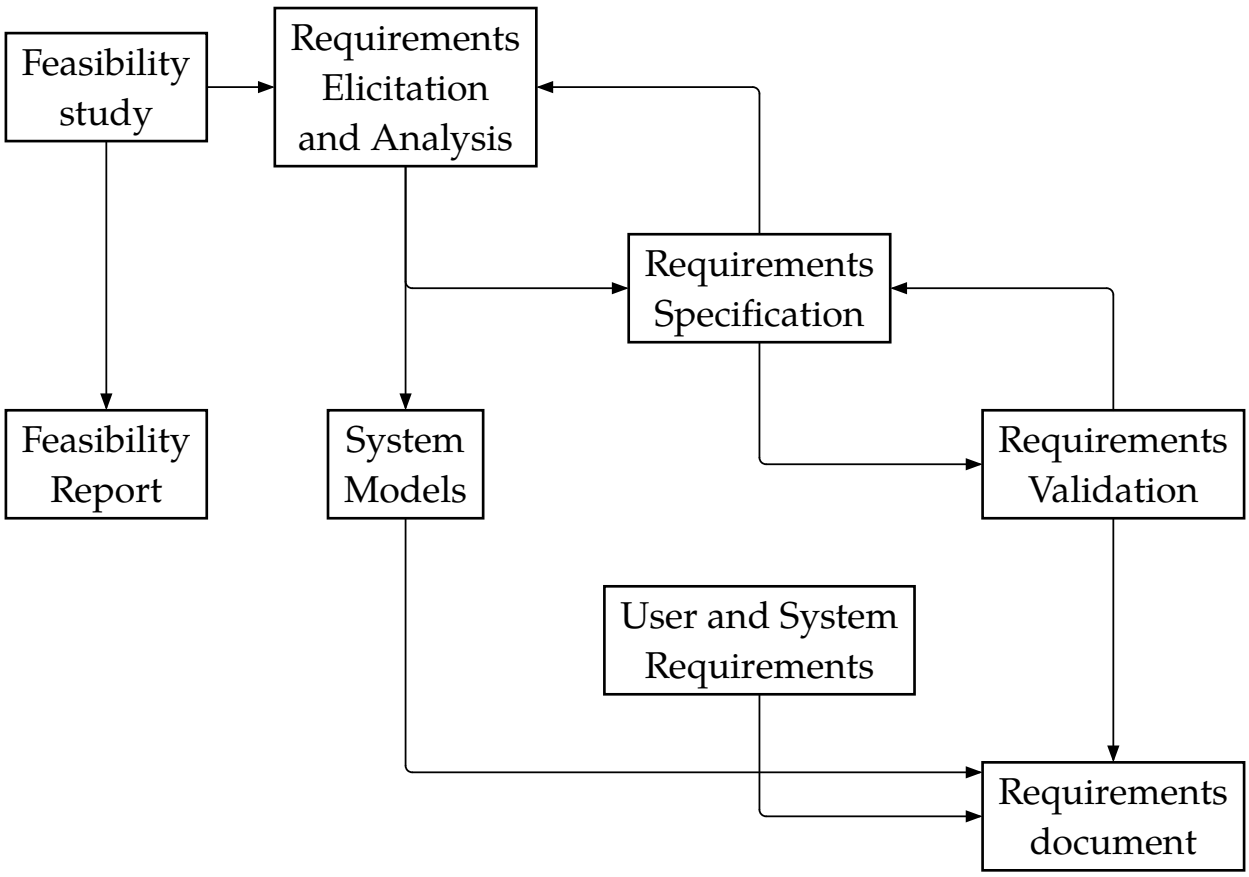

Figure 2: Requirements Engineering process

and has some activities:

Activity	**Function**
Feasibility study	Determine if the **user needs can be satisfied** with the available technology and budget.
Requirements elicitation	Find out what **system stakeholders** require from the system
Requirements analysis	**Clarify/define/document the requirements** in a form understandable to the customer
Requirements specification	**Define the requirements** in detail. (Written as a contract between client and contractor.)

1.3.1. Requirements Elicitation

Sometimes called **requirements discovery**, it involes expert working with customers to define **domain**, **services** (functionality + quality), and

operational **constraints**. It involves various **stakeholders**, such as end-users, managers, engineers, domain experts, etc.

1.3.2. Requirements Analysis

Requirements Analysis require **collaboration** of people with different backgrounds, such as **users** with domain knowledge, and **developers**, and it bridges the gaps between them.

2. Software design

> The design of a system determines a set of components and inter-component interfaces that satisfy a specified set of requirements.

Software Design can be seen as **translating and refining** a (problem) domain model into abstractions that can be directly mapped to an implementation in a programming language.

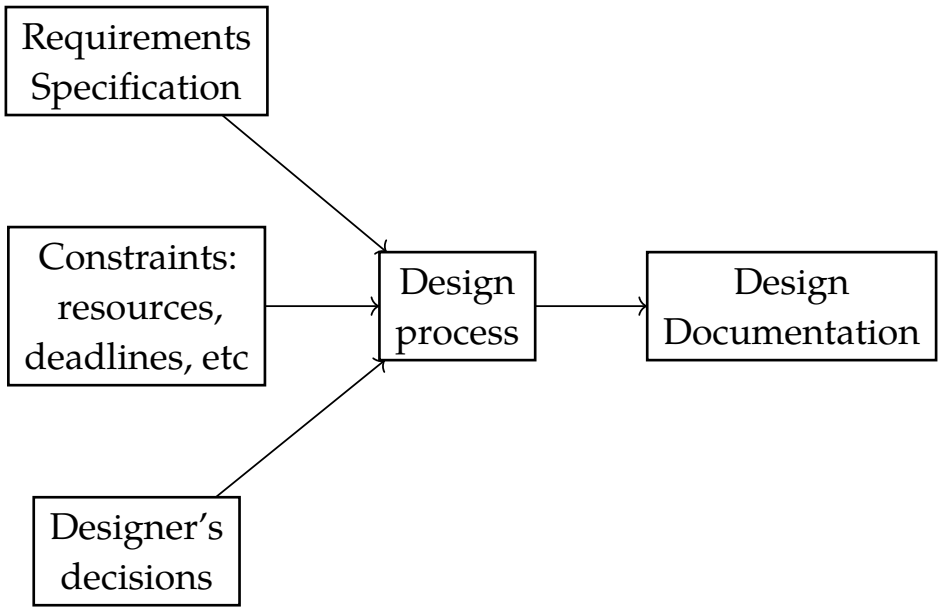

Figure 3: Software Design Process

Design Patterns conduct standard solutions to common software design problems.

3. Abstractions and Models

3.1. Models

Models are **simplification of reality**. Models are created to better understanding of the problem, the domain, the system to be built.

A model can capture and represent:

- Structure (static model)
- Behavior (dynamic model)

Example:

- Architectural blueprint
- Drawn UI
- Class diagrams

3.2. Abstractions

Abstractions are **concepts or ideas** not associated with any specific instance. It leaves out irrelevant detail, and highlights important events.

Some examples of abstractions:

- Instruction
- Condition
- Subsystems
- Packages
- Interfaces
- Abstract data types
- etc

3.3. Models in abstractions

Models are abstractions, in the sense that they are created for better understanding.

A domain model is a model that **captures** the **domain entities / concepts relevant to the original problem**. A **domain vocabulary** defines the meaning / interpretation of domain entities. The relationships between domain entities define an *abstraction over the problem domain*.

4. Scenarios and use cases

4.1. User perspective

Users tend to think about systems in terms of "features" (what system can do), e.g. the system must provide a function to X. They should tell **stories** involving those features, and **scenarios** and **use cases** can assist to get requirements complete and consistent.

Scenarios: Users interact with a computer systems to complete a **task** or achieve a **goal**, which can be captures as a set of scenarios, i.e.

> Mary browses the catalogue and adds desired items (chocolate) to the shopping basket. When she has finished shopping, Mary provides her address for delivery, credit card information and then confirms the transaction. The system confirms her payment and e-mails a transaction record.
>
> — Example user story

We note that we covered the **ideal interaction** (everything works properly). But:

- What happens if the credit-card payment fails?
- What if the customer is a repeat customer, so shipping details are already in the system?

The key scenario remains, but to answer the questions we have to add *alternative* scenarios.

Scenarios can also be written down as a series of steps, and we need to identify scenarios that can be written up.

4.2. Use cases

If we structure scenarios, they can represent the behavioural requirements of the system, *from the users' viewpoint*. Every use case is directly or indirectly *invoked by an actor*, and must be *tied to a specfic goal*.

4.3. Actors

An **actor** is a **role the user plays** when they interact with a system. They *perform* use cases. Actors can have multiple use cases, and single use case may have several actors.

Actors don't have to be human.

5. Goal-Design Scale (unclear why a scale, but ok)

- *Goal Level* requirements satisfy business goals, e.g. calculations hit 5%
- *Domain Level* requirements satisfy the activities that go on outside the product, e.g. products support cost recording and quotation with experience data, e.g. product supports cost recording and quotation with experience data
- *Product Level* requirements specify what should come in and out of the product, e.g. product must have recording and retrieval function for experience data
- *Design level* requirements specify interface details between the system and other entities, interacting with it (such as users and other systems) and internals of the system, e.g. System shall have sreen pictures

6. Domain Model Development

Also called *Domain Modelling*, the process is as follows:

1. Perform a **textual analysis** of the problem by understanding the problem domain; *circling all nouns* and *underlining all verbs*
2. Write down relevant **domain entities**, which is normally some of the nouns defined above
3. Revisit the specification to extract the **associations**
4. Record **associations** with the corresponding **domain entities**

7. Task Descriptions

Each Task Description has:

- **Purpose**: What the task aims to **achieve**
- **Trigger**: The **event that starts** the task
- **Precondition**: The condition that allows the task to occur
- **Frequency**: **How often** the task occurs

  - **Critical**: Special **high-pressure or large-scale** situations
- **Subtasks**: A list of steps that make up the task
  - Written in **neutral language**.
  - Describe what the user and computer must accomplish *together*.
- **Variants**: **Special cases or alternative ways** to perform subtasks.

Good tasks should have:

- All events covered
- Critical tasks covered

8. Workflows

A **workflow** is a coherent representation of the (**temporal**) dependencies between *the major steps of a project*. It focuses on:

- Actors of each step
- Dependencies between steps
- *Sequencing*

Workflows assist in identifying *high-level* tasks and a sequence of *good* user tasks.

9. Object Oriented Design

Object Oriented Design (*OOD*) is a method of design encompassing the process of **object-oriented decomposition**. It views the system as *interactions of different objects*.

The goal of *OOD* is to find the important **object-based abstractions** of the system. We should ask questions:

- How can we **identify classes/objects**?
- How do objects **interact**?
- What **roles and responsibilities** do objects perform?.

9.1. Objects and classes

Objects are an **abstract model of a real-world entity** that has an independent **identity**. In design and implementation, it is a **component** that *contains a collection of data and related procedures*.

Objects can also **receive** and **response** to messages, and has behavior that may be dependent on its internal state, (the state can change by responding to messages).

Classes is a **description** of a **group of objects** with common data attributes, common operations, common relationships and semantics. Classes can be used to *create* objects called **instances**. A class should also capture **one and only one** key abstraction. If this is violated, it reduces *cohesion* (see Section 9.10).

Do not create god classes/objects. They have many responsibilities, with significant complexity and difficult to understand and maintain.

Classes can be described/represented in *UML*, using Mermaid Diagrams, i.e.:

```
classDiagram
    class Rectangle{
        - width: int
        - length: int
        + calculate_area() int
        + calculate_parameter() int
    }
```

where the plus signs mean public attributes and the negative signs mean private attributes.

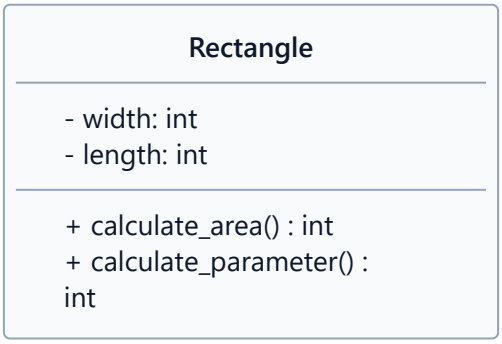

Figure 4: Sample UML diagram

9.2. Object-Oriented Design Process

The process then works with:

1. Find the **classes**
2. Determine the **responsibilities** of each class, and study the requirements specficiation;

3. Determine **how objects collaborate** to fulfill their responsibilities

Then

4. **Factor** common responsibilties to build class hierarchies
5. **Streamline collaborations** between objects.
   - e.g. is message traffic heavy in parts of the system?
6. Apply **design heuristics** to improve specific design aspects

The result in the design process is **not** a final product. Design decisions may be revisited, and it is rarely linear. Furthermore, design methods provide *guidelines*, not fixed rules. A good *sense of style* often helps produce *clean, elegant designs*, a.k.a designs that make sense from the engineering standpoint.

9.3. Finding classes

Identify:

- What are the **goals** of the system being designed, and the expected **inputs** and **outputs**?
- Look for **noun phrases**, separate into **obvious** classes, **uncertain** candidates, and **nonsense**.
  - Example noun phrases: **physical objects**, **conceptual entities**, etc.

CRC Cards can help with finding classes.

9.3.1. CRC Cards

CRC (*Class, Responsbility and Collaborations*) cards are used to capture and organize information about classes and responsibilities within a software system

Animal	
Responsibility	**Collaboration**
Model The Animal	Zoo

Table 2: Example CRC Card

CRC cards are used to **explore** possible designs, and **are not specification of designs**.

9.4. Responsibilities and Assigning Responsibilties

9.4.1. What are responsibilities?

Responsibilities are the **knowledge** an object maintains and provides, and the **actions** it can perform. It represents the **public services** an object may provide to client. Namely, it states what it does, **not** how it does.

To identify responsbilities, study the requirements specification, namely:
- Highlight **verbs**,
- Explore **alternative scenarios**
- Identify **actions** resulting from **different inputs**

Furthermore:
- Study classes, and use class names to infer roles and responsibilities.
- Use recorded purposes on cards (CRC cards?) as responsibilities

9.4.2. Assigning Responsibilities

Use Pelrine's Laws:
- "Don't do anything you can push off to someone else."
- "Don't let anyone else play with you"

which means:
- **Evenly distribute** system intelligence (or *work*) *uniformly* among **top-level classes**:
  - avoid **centralization**, and keeps responsilbilities close to objects.
- State responsibilities as **generally** as possible
- Keep **behavior** together with related information.
- Keep information about **one thing** in **one place**. If multiple objects need to access the same information, we need a **new object to manage**, **one obvious candidate**, or **collapse**
- **Break down complex responsibilities** and *share* responsibilities among related objects.
- **Maintaining information** *is* a respobsibility

9.5. Relationships between classes

9.5.1. Example relationships

Additional responsibilities can be uncovered by examining relationships between classes.

Relationship Type	Definition
Is-Kind-Of	Classes sharing a **common attribute** often share a **superclass** ⇒ common responsibilities
Is-Analogous-To	**Similarities** between classes can suggest undiscovered **superclasses**
Is-Part-Of	**Distinguish** responsibilities between *part* and *whole*

Table 3: Some kinds of relationships between classes

9.5.2. Association and Aggregation (and Composition)

- **Composition** is a "part-of" relationship where the **lifecycle of the contained object(s)** are is **completely dependent on the container object**.
- **Aggregation** is a "has-a" relationship where **the contained object exists indepdently of the container**.
  - It is useful for building **modular and reusable** components.
- **Association** is a relationship between **two independent classes**, where an object uses or interacts with other object(s).
  - Associations can be either *unidirectional* or *bidirectional* and exists in forms as *one-to-one*, *one-to-many*, *many-to-one* and *many-to-many*

For good design, whenever there is a choice in an OOD between a **containment relationship** and an **association relationship**, pick the former, since it reduces amount of exposure of internals of a class[1]

9.5.3. Collections

The elements of an assocation / aggregation generally needs some form of expandable container / collection in the corresponding enclosing object. Some notes of how to design collections:

- If you want to **avoid duplicates**, choose a **set-like collection**.
- If you want to **access elements by index**, choose a *indexable* collection (i.e. Array)
- If you want to **have a specific element order**, choose a collection that **allows ordering**.
- If you want some of the above, choose the collections that **supports all**.

[1] something something encapsulation

9.6. Inheritance and Abstract Classes

9.6.1. Inheritance

Model Inheritance reflects "is-a" relations between abstractions in the model. **Software Inheritance** expresses relations between the software system.

Inheritance should be used **only** to model a **specialization hierarchy** (which classes are speicalizations of others?)

Theoretically, inheritance hierarchies can be *really* deep, but in practice, they are rarely deep, and should be no deeper than the short-term memory of a developer (5-7 layers).

9.6.2. Abstract Classes

Abstract Classes are classes that factor common behavior shared by other classes. It groups **related classes** with common responsibilities, and help introduces parent classes that represents the group.

9.6.3. How to design abstract classes

Good candidates for abstract classes are *categories*.

All abstract classes must be *base* classes (other classes derive from them). and all base classes should be abstract classes.

Concrete classes may be both instantiated and inherited from, while abstract classes can only be inherited from (i.e. **no** inheriting abstract classes from concrete classes). Sub-classes should **support all inherited responsbilities** (and possibly more). They may refine or specialize a responsibilitity, but never hide them from the parent class.

9.7. Colaborations

In OOD, objects **collaborate** to perform the desires responsbilities.

For each responsibilitity, ask whether the class can fulfill the responsibility **by itself**, and **if not**, then:

- what does it **need**?
- From what *other* class can it obtain that?

For each class, ask:

- What does it **know**?

- What *other* class(es) need its **information / results**?

Some interaction considerations:
- **Minimize** the number of classes with which another class collaborates (i.e. don't have way too many collaborators for one class).
- **Minimize** the amount of collaboration between a class and its collaborators, (minimize the **number of messages sent**).
- **Do not** turn an operation in to a class.
- Classes that **do not interact** with others should be discarded.

9.7.1. Roles and Protocols

Collaborations are often described using **protocols**. Each participant in a protocol play a given **role**.

For example:

> **Drawing Element** is a class of the graphics editor. It defines three *roles*:
> 1. *Client*: Uses other elements of the application
> 2. *Child*: Can be contained in a Group Element
> 3. *Subject*: state being observed by other elements.
>
> Each of these roles implies a number of **related respobsibilities**, and the usage of a particular **collaboration protocol**.

9.8. Design by Contract

Design by Contract is a design concept introduced by Bertrand Mayer in the late 1980s. It views the relationship between a **service provider** (an object) and **service consumer** (a *client* of the object) as a **formal agreement** (a *contract*) with rights and obligations.

It extend the definition of abstract data types (i.e. *classes*) with **pre-conditions, post-conditions, invariants**, and how they can be refined in classes.

In DbC:

- A *service consumer* must ensure that a service on a provider is only ever invoked **when the service's preconditions are met**.

- A *service provider* must ensure that **after successful completion of a service, the postconditions are met**.
- Invariants of the service provider must be held **after the successful execution** of any externally visible service(s).
- Subclasses can only **weaken service pre-conditions** or **strengthen service post-conditions**

9.8.1. Abstract State v.s. Concrete State.

Every object contains **stateful information** (i.e. objects have **state**).

The externally visible properties of this state defines the **abstract state**. The internal representations define the **concrete state**[2].

Clients should only rely on an objects' abstract state, and similarly object equality should only ever be defined over the abstract state.

9.8.2. Command-Query seperation principle.

All methods of a class should be divided into two non-overlapping categories:

Queries	Commands
Returns a result, **without changing the observable state**	**Changes the (abstract) state** without returning a result.

Table 4: Command-Query Seperation Principle

For example:

```
>>> a = [1, 2, 3, 4] # object declaration
>>> a.__len__() # query, returns a result, doesn't change the
object
4
>>> a
[1, 2, 3, 4]
>>> a.append(5) # command, doesn't return a result, changes the
object
>>> a
[1, 2, 3, 4, 5]
```

[2]So it seems like *abstract state* = *public* and *concrete state* = *private*. What the hell happened to *protected*?

9.9. Decomposition

Decomposition (also called *divide-and-conquer*) is a process that helps to **manage the complexity** of a system.

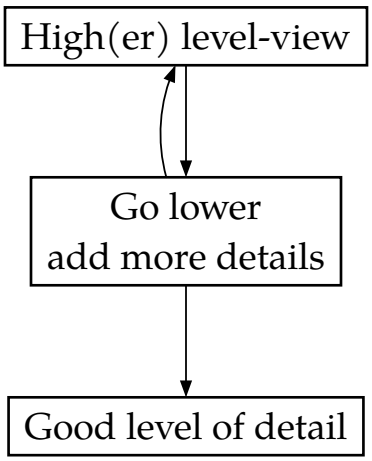

Figure 5: Decomposition Process

There are two types of decomposition: *functional* and *object-oriented* decomposition.

9.9.1. Functional v.s. Object-oriented Decomposition

Type	How to Decompose?	Issues
Functional	Decompose according to **functions** a system should perform	• Naive: modern systems perform **more than one function** • Maintainability: System functions evolve, design affects whole system • Interoperability: Interfacing with other system is difficult
Object-oriented	Decompose according to the **objects** a system is supposed to manipulate	Needs to identify objects

Table 5: Functional v.s. Object-oriented Decomposition

9.10. Coupling and Cohesion

Coupling and Cohesion are **measures** of **how well the parts of a system are tied together** to form the system itself.

Measure	Definition
Coupling	**inter-dependency** between system parts
Cohesion	**intra-dependency** of a system part

Table 6: Coupling and Cohesion

The goal of good OOD is to have **weak coupling** (system parts are independent and changing one shouldn't change others) and **strong cohesion** (code is tightly related, each module has one purpose)